

Introduction to Databases and the Relational Model

Databases and Software Architecture – Aalborg University

Claude, a large language model developed by Anthropic, was used to assist with structuring and grammar checking of this material. All generated text was critically reviewed, edited, and verified by the author.

Introduction to Databases and the Relational Model

This is the opening lecture of the course, introducing the relational model and the modelling techniques used throughout the course. Today lays the foundation for everything that follows: why we use databases at all, how the relational model represents data as tables, what keys are, and how to write your first SQL statements against a real PostgreSQL database.

Agenda

- Motivation: DBMS vs. file-based storage
- Data models and the relational model
- Keys and integrity rules
- Break
- Live demo: PostgreSQL, Student & Programme
- Questions, assignment, and the in-class exercise

Motivation

Where Is Your Data Right Now?

Think about any application you use every day – a webshop, a bank, a student portal. Every one of them needs to store, retrieve, and update data reliably, often for years at a time. Before we talk about databases formally, it's worth asking: if you had to build one of these from scratch tomorrow – in whatever language you like, Java, Python, C#, or otherwise – how would you even store the data so that it's still there the next time the program runs?

Where Can I Find Databases?

Take thirty seconds: name three apps or devices you have used *today* that are, almost certainly, backed by a database.

Databases Are Everywhere

The honest answer: almost everywhere you look.

- **Phones** – contacts, photos, and messages typically live in a small on-device database (often SQLite)
- **PCs** – installed apps and browsers store your history, bookmarks, and passwords locally
- **Servers** – web apps, e-commerce stores, and banking backends are clients talking to a database server
- **Everywhere else** – cars, ATMs, smart thermostats, your streaming watch-history, your game saves – all quietly backed by a database

By the end of this course, all of these will look a lot less mysterious.

The File-Based Approach

The most naive answer to “how do I store this?” is to write everything to plain files – a `.txt` or `.csv` per “table.” The program opens the file, reads its contents into memory, processes them, and writes the result back before it exits. That approach genuinely works for a small toy program. But it breaks down quickly once multiple people need to use the data at the same time, once the data has real relationships to track (students take courses, orders contain items), or once correctness actually matters – think money, grades, or medical records.

Problems with File-Based Storage (1/2)

Push the file-based approach a little further and the same problems show up, every time:

- **Redundancy** – the same fact ends up copied into multiple files, or multiple places within one file. A student's email might get written once for every course they are enrolled in, wasting storage and creating copies that all have to be kept in sync by hand.
- **Inconsistency** – redundant copies drift apart. Update a student's email in `course101.txt` but forget `course202.txt`, and the “truth” now depends entirely on which file you happen to open.

Problems with File-Based Storage (2/2)

- **No access control** – plain files have no built-in notion of who is allowed to see or edit what. Anyone with file access can read – or corrupt – the whole thing, when really only teachers should enter grades and students should only see their own data.
- **No concurrent access** – two people open the same file, both edit it, both save; whichever save happens last silently overwrites the other's changes. Files were never designed for many simultaneous readers and writers to coordinate safely.

Hand-rolling solutions to all four of these problems yourself is exactly the kind of hard, error-prone work we would rather not repeat in every project.

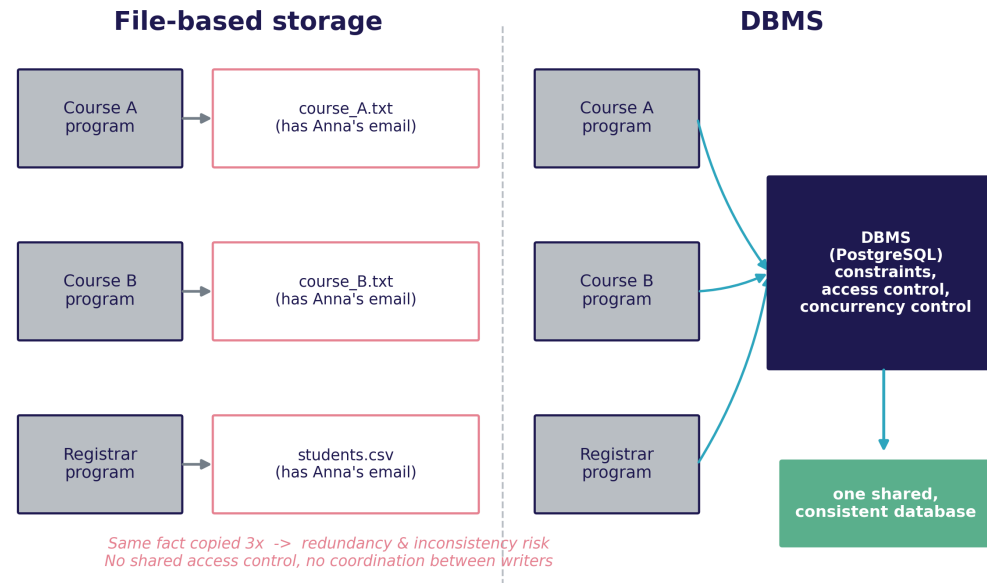
Enter the DBMS

A Database Management System, or DBMS, is software that manages all of this for you, out of the box:

- a structured way to describe your data – a **data model**
- a query language to retrieve and change that data
- mechanisms that actively guard against redundancy and inconsistency
- **access control** for different users
- safe **concurrent access**, through transactions, which we cover properly later in the course

In short: you stop reinventing all of this by hand, in every application you write, regardless of language.

Files vs. a Shared Database



Duplicated files vs. one shared, DBMS-managed database

Database vs. DBMS vs. Database System

Three related terms are worth pinning down precisely, because we will use all three throughout the course:

- **Database** – a collection of related data that models some part of the real world
- **DBMS** – the software that lets you define, create, query, update, and administer databases
- **Database System** – the combination of a database *and* the DBMS managing it

Examples of DBMS software you will encounter in practice include PostgreSQL, MySQL, Oracle, SQL Server, and SQLite.

Why This Matters for Software Architecture

Where and how an application stores its data is not a minor implementation detail – it is an architectural decision. It affects correctness, performance, scalability, and how the rest of the system ends up structured, and we will come back to this explicitly when we cover the Repository and DAO patterns later in the course. This course starts at the foundation: the relational model and SQL.

Data Models and the Relational Model

Data Models: A Brief History

A data model is a collection of abstract concepts for describing data – essentially, the theory that dictates how a database is structured. Databases did not start out relational: the field went through three major eras.

- **Hierarchical model** – 1960s
- **Network model** – late 1960s to 1970s
- **Relational model** – 1970 onward

The relational model is what virtually every mainstream database – including PostgreSQL – still uses today.

Hierarchical Model

In the hierarchical model, data is organized as a tree: every record has exactly one parent. A Department has many Employees, and each Employee has many Payslips, for example. That structure is fast to navigate along that one fixed path, but it is rigid – it struggles to represent data that naturally has more than one parent, such as a student enrolled in two courses at once.

The network model generalizes the tree into a graph, so a record can have multiple parents rather than just one. That is more flexible than the hierarchical model, but navigation is still done by following explicit, hand-coded pointers between records. Both of these earlier models require the programmer to know the physical structure of the data just to query it.

The Relational Model

The relational model was proposed by E. F. Codd at IBM in 1969¹, and its key idea was to represent all data uniformly as simple tables. Crucially, it separates the logical view of the data – the tables you see – from the physical storage underneath – the files, indexes, and disk layout the DBMS actually uses. That separation is exactly why SQL is a declarative language: you describe the result you want, and the DBMS figures out how to get it, rather than you writing out the navigation path yourself.

¹Codd, E.F. (1969), *Derivability, Redundancy, and Consistency of Relations Stored in Large Data Banks*, Research Report, IBM. A refined version was published the following year as “A Relational Model of Data for Large Shared Data Banks,” *Communications of the ACM*, 13(6), 377–387 (1970) – the paper most often cited as introducing the relational model.

A Table Is a Mathematical Relation

Formally, a table is a relation: a subset of the Cartesian product of a set of domains, or value ranges. Practically, it is easier to think of a table as rows and columns – one row per real-world fact or entity instance, and one column per property recorded about each instance. You will hear both vocabularies throughout the course: the formal terms relation, tuple, and attribute mean exactly the same thing as the everyday terms table, row, and column.

Running Example: Student and Programme

We will use this example for the rest of the lecture – and build it live in PostgreSQL right after the break.

Student

student_id	first_name	last_name	email
1	Anna	Jensen	anna.jensen@student.aau.dk
2	Mikkel	Sørensen	mikkel.sorensen@student.aau.dk

Programme

programme_id	programme_name	duration_years
1	Software Engineering	3
2	Computer Science	2

Tuple, Attribute, Domain

Three terms describe the anatomy of a table in detail:

- **Tuple** – one row: one ordered set of values, corresponding to one specific student
- **Attribute** – one column: a named property, such as `email`
- **Domain** – the set of legal values an attribute is allowed to take (`duration_years` must be a positive integer, `email` must look like an email address, `student_id` must be a positive integer too)

Every value stored in a table has to come from the declared domain of its attribute – that is the DBMS's first line of defense against bad data.

Anatomy of the Student Table

Student

<- relation / table name

student_id	first_name	last_name	email
1	Anna	Jensen	anna.jensen@student.aau.dk
2	Mikkel	Sørensen	mikkel.sorensen@student.aau.dk
3	Freja	Nielsen	freja.nielsen@student.aau.dk

attribute (column):
← a named property,
shared by every row

tuple (row):
← one student -
one ordered set of values

domain: the set of legal values for
an attribute (student_id: positive integers)

Anatomy of the Student table: table name, attribute (column), tuple (row), and domain, labelled on one concrete example

Schema vs. Instance

It is worth separating two things that are easy to conflate. The **schema** is the structure – the table names, attribute names, and their domains – and it rarely changes once a system is running. The **instance** is the actual data sitting in the tables at a given moment, and it changes constantly, as new students enroll every semester. A useful analogy from object-oriented programming – whether that is Java, Python, or C#: the schema is like a class definition, and an instance is like the objects created from that class at runtime.

Two more terms round out the vocabulary. The **degree** (or arity) of a relation is the number of attributes it has – `Student(student_id, first_name, last_name, email)` has degree 4. The **cardinality** of a relation is the number of tuples it currently holds – if `Student` has 30 rows right now, its cardinality is 30. Cardinality changes constantly as data is added and removed; degree only changes when you deliberately alter the schema.

Terminology Recap

Formal term	Everyday term
Relation	Table
Tuple	Row / Record
Attribute	Column / Field
Domain	Value range / type
Degree	Number of columns
Cardinality	Number of rows

Keys and Integrity Rules

Why Do We Need Keys?

A table is technically just a set of tuples, but in practice we constantly need to answer questions like “give me the one row for student Anna Jensen.” The trouble is that two students could easily share the same name, so names alone cannot reliably identify a row. Keys are how the relational model guarantees that we can always uniquely identify – and refer back to – one specific tuple.

Superkey

A **superkey** is any set of attributes that uniquely identifies a tuple in a relation – no two rows can ever share the same combination of values for those attributes. For Student:

- {student_id} is a superkey
- {student_id, email} is also a superkey
- {student_id, first_name, last_name} is also a superkey

Once a set of attributes is already unique, adding more attributes on top of it is still unique – so it is still a superkey. That is why a relation can have many superkeys at once.

Candidate Key

A **candidate key** is a *minimal* superkey – remove any single attribute from it and it stops being unique.

- {student_id} on its own **is** a candidate key for Student
- {student_id, email} is **not** a candidate key: it is a valid superkey, but not minimal, since {student_id} alone already does the job
- {email} could also qualify as a candidate key, provided every student has a unique, non-null email address

A table can have several candidate keys at once; the database designer then picks one of them to serve as the primary key.

Primary Key

The **primary key** is whichever candidate key gets chosen as the main identifier for a table, and every table should have exactly one. Once you declare a primary key, the DBMS automatically enforces two guarantees on it:

- **uniqueness** – no two rows may ever share the same primary key value
- **not-null** – a primary key value must always be present

For our running example, `Student.student_id` and `Programme.programme_id` will be our primary keys.

Foreign Key

A **foreign key** is an attribute – or set of attributes – in one table that refers to the primary key of another table (or, occasionally, the same table). It is how the relational model represents relationships between tables without duplicating data.

A foreign key does not strictly have to reference a primary key – any UNIQUE-constrained column works, since guaranteed uniqueness is the real requirement. Every foreign key in this course references a primary key, the common case.

In our example, `Student.programme_id` refers to `Programme.programme_id`: instead of copying the full programme name and duration into every single student row, we just store a reference to the programme they are enrolled in.

Preview: Why Normal Forms Exist

Splitting data into separate, linked relations so each fact – a programme's name, a student's email – is stored exactly once is not just good taste: it is precisely the design goal that **normalisation** formalizes. A later lecture has an optional deep-dive on this, once you have seen `CREATE TABLE` and foreign keys in full.

Writing It Down: Relation Schema Notation

A full CREATE TABLE is often more than you need just to **sketch** a design on paper. Throughout this course's exercise sheets, you will see a compact shorthand for a relation schema instead: its name, its attributes and their types, the primary key underlined, and an arrow marking a foreign key.

```
Programme: {[ programme_id: int, programme_name: string, duration_years: int ]}
```

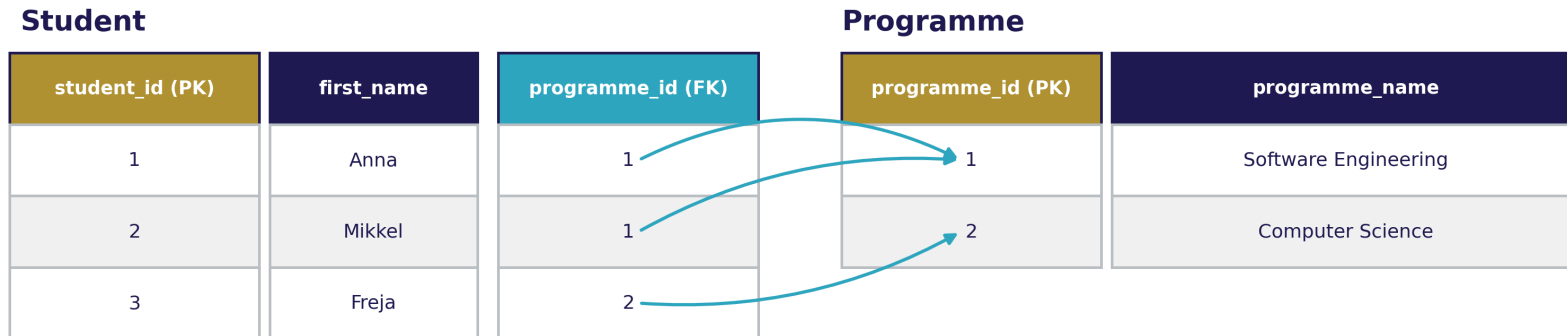
```
Student: {[ student_id: int, first_name: string, last_name: string, email: string, birth_date: date, programme_id → Programme ]}
```

- **Underline** – primary key
- **→ RelationName** – foreign key, referencing that relation's primary key (the common case)
- No underline, no arrow – an ordinary attribute

Student and Programme, with Keys Marked

`Student.programme_id = 1` points to the row in `Programme` where `programme_id = 1` – and that single link is exactly what lets us avoid repeating “Software Engineering, 3 years” in every student row enrolled in that programme.

Student and Programme: Keys Linked



*Student.programme_id (FK) points to Programme.programme_id (PK)
Referential integrity: every FK value must match an existing PK value in Programme*

Student.programme_id (FK) referencing Programme.programme_id (PK), with matching rows connected

The first of three integrity rules the DBMS enforces automatically:

- **Domain integrity** – every attribute value must belong to its declared domain, or type

This is enforced directly by the column type you declare – `INTEGER`, `VARCHAR(50)`, `DATE`, and so on. Try to insert the text "three" into a `duration_years` `INTEGER` column, for instance, and the DBMS rejects it outright, before it ever has a chance to become bad data sitting in your table.

The second integrity rule:

- **Entity integrity** – every table's primary key must be unique and not null, for every single row

This guarantees that every tuple can always be unambiguously identified. Without this rule, `student_id` could end up missing or duplicated on some row, and a query like “give me the row for student 7” would no longer have one clear answer.

The third integrity rule:

- **Referential integrity** – every foreign key value must either match an existing value in the (primary- or unique-keyed) column it references, or be NULL, if the relationship allows that

This guarantees you can never end up with something like `Student.programme_id = 99` pointing at a programme that does not exist – no dangling references. Once you declare the foreign key constraint, the DBMS enforces this automatically, on every insert and update.

Keys and Integrity: Recap

Before the break, here is the whole vocabulary in one place:

- **Superkey** → uniquely identifies a row (may include extra, unnecessary attributes)
- **Candidate key** → a minimal superkey
- **Primary key** → the candidate key chosen to identify a table
- **Foreign key** → a reference from one table to another table's primary key (or a unique column)
- **Domain / entity / referential integrity** → the three rules the DBMS enforces automatically, so your data can never silently drift into inconsistency

Break



5-Minute Break

Take five minutes – stretch, grab some water, and feel free to ask a quick question if something from the first half is still unclear. When we come back, we switch from theory to practice: we will build `Student` and `Programme` for real, from scratch, in PostgreSQL.

Live Demo

Live Demo: What We'll Build

For the next block, we build a real PostgreSQL database containing two tables, Programme and Student, with `Student.programme_id` as a foreign key referencing `Programme.programme_id`: each student is enrolled in exactly one programme, and each programme has many students – a genuine one-to-many relationship. We will create the tables, insert some rows, and then run a sequence of `SELECT` queries of increasing complexity. Follow along on your own machine if you already have PostgreSQL installed; otherwise, just watch.

Step 1: Create the Database

Run this once, from `psql` or a GUI client such as pgAdmin or DBeaver, to create the database we will work in for the rest of the demo – then connect to it before running anything below.

```
CREATE DATABASE dbs_lecture1;
```

```
\c dbs_lecture1
```

Step 2: CREATE TABLE Programme

```
CREATE TABLE Programme (  
    programme_id    SERIAL PRIMARY KEY,  
    programme_name  VARCHAR(100) NOT NULL,  
    duration_years  INTEGER NOT NULL  
);
```

SERIAL gives us an auto-incrementing integer, so PostgreSQL fills in `programme_id` for us automatically. PRIMARY KEY enforces entity integrity without any extra work on our part, and the NOT NULL constraints on `programme_name` and `duration_years` say that both of those values are always required.

Step 3: CREATE TABLE Student

```
CREATE TABLE Student (  
    student_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name  VARCHAR(50) NOT NULL,  
    email      VARCHAR(100) UNIQUE NOT NULL,  
    birth_date DATE,  
    programme_id INTEGER NOT NULL REFERENCES Programme(programme_id)  
);
```

The interesting line here is `programme_id ... REFERENCES Programme(programme_id)` – that is the foreign key declaration itself.

Step 4: Insert Data into Programme

```
INSERT INTO Programme (programme_name, duration_years) VALUES  
    ('Software Engineering', 3),  
    ('Computer Science', 2),  
    ('Data Science', 2);
```

Notice that we do not specify `programme_id` in the insert – PostgreSQL assigns it automatically, in order, as 1, 2, and 3.

Step 5: Insert Data into Student

```
INSERT INTO Student (first_name, last_name, email, birth_date, programme_id) VALUES
  ('Anna', 'Jensen', 'anna.jensen@student.aau.dk', '2004-03-12', 1),
  ('Mikkel', 'Sørensen', 'mikkel.sorensen@student.aau.dk', '2003-11-02', 1),
  ('Freja', 'Nielsen', 'freja.nielsen@student.aau.dk', '2004-07-19', 2),
  ('Oliver', 'Christensen', 'oliver.christensen@student.aau.dk', '2004-01-25',
3),
  ('Sofie', 'Larsen', 'sofie.larsen@student.aau.dk', '2003-09-08', 2);
```

Step 6: A Simple SELECT

```
SELECT * FROM Student;
```

The asterisk means “all columns,” and FROM Student means “read from the Student table.” With no filtering condition at all, this returns every single row.

Step 7: Filtering with WHERE

```
SELECT first_name, last_name  
FROM Student  
WHERE programme_id = 1;
```

Now we pick only the columns we actually want, and only the rows that match a condition. Read aloud, this says: give me the first and last name of every student enrolled in programme 1.

Step 8: Sorting with ORDER BY

```
SELECT first_name, last_name, birth_date  
FROM Student  
WHERE programme_id = 1  
ORDER BY birth_date;
```

ORDER BY sorts the result set – ascending by default – and, as this query shows, WHERE and ORDER BY combine freely in the same statement.

Step 9: A Taste of JOIN

```
SELECT s.first_name, s.last_name, p.programme_name  
FROM Student s, Programme p  
WHERE s.programme_id = p.programme_id;
```

This combines rows from two tables at once, using the foreign key relationship between them. Read aloud: for every student, look up the name of the programme they are enrolled in. Consider this a preview – the proper syntax and theory behind JOIN is coming later in the course.

Demo Wrap-Up: What's Coming

Today we covered `CREATE TABLE`, `INSERT`, and simple `SELECT` with `WHERE` and `ORDER BY` – enough to start experimenting on your own. Here is where each piece gets developed further:

- Single-table queries and aggregation, in depth
- Joins and subqueries, properly
- `UPDATE`, `DELETE`, and data integrity in more depth
- Doing all of this from application code (JDBC in Java, this course's concurrent track)

Wrap-Up

Questions?

This is everyone's first real exposure to databases, so no question is too basic – whether it is about files versus a DBMS, tables and tuples and attributes, or keys. If you think of something later contact details are on Moodle.

Today we covered why DBMSs exist in the first place, the relational model – tables, tuples, attributes, and domains – keys, the three integrity rules, and your first SQL.